

Question No:2

Problem Description

You are given an N-dimensional array A. You can execute an operation that removes the greatest and smallest elements from the array and replaces them with their difference. As a result, the array's size will drop by one after each operation. You are assigned Q jobs, each of which contains an integer K. After K operations, you must provide the total of all the items in the array for each task.

Constraints:

$$2 \leq N \leq 10^5$$

$$1 \leq Q \leq 10^5$$

$$0 \leq A[i] \leq 10^9$$

$$0 \leq K < N$$

Input:

First line contains two space-separated integers N and Q, denoting the number of elements in array and number of queries respectively.

Next line contains N space-separated integers denoting elements of the array.

Next Q lines contain a single integer K.

Output:

For each task, print answer in a new line.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 2
3 2 1 5 4
1
2
```

EXPECTED OUTPUT

```
13
9
```

Test Case 2

INPUT (STDIN)

```
5 3
13 12 11 15 14
1
2
3
```

EXPECTED OUTPUT

```
43
35
15
```

Code:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAXN 100005
5
6  typedef long long ll;
7
8  typedef struct {
9      ll value;
10     int id;
11 } Node;
12
13 Node minHeap[2 * MAXN];
14 Node maxHeap[2 * MAXN];
15
16 int minSize = 0;
17 int maxSize = 0;
18
19 int active[2 * MAXN];
20
21 /* ----- MIN HEAP ----- */
22
23 void minPush(Node x)
24 {
25     int i = ++minSize;
26
27     while (i > 1 && minHeap[i / 2].value > x.value)
28     {
29         minHeap[i] = minHeap[i / 2];

```

```
29         minHeap[i] = minHeap[i / 2];
30         i /= 2;
31     }
32
33     minHeap[i] = x;
34 }
35
36 Node minPop()
37 {
38     Node result = minHeap[1];
39     Node last = minHeap[minSize--];
40
41     int i = 1;
42
43     while (i * 2 <= minSize)
44     {
45         int child = i * 2;
46
47         if (child + 1 <= minSize &&
48             minHeap[child + 1].value < minHeap[child].value)
49         {
50             child++;
51         }
52
53         if (minHeap[child].value >= last.value)
54             break;
```

```
56         minHeap[i] = minHeap[child];
57         i = child;
58     }
59
60     if (minSize >= 0)
61         minHeap[i] = last;
62
63     return result;
64 }
65
66 /* ----- MAX HEAP ----- */
67
68 void maxPush(Node x)
69 {
70     int i = ++maxSize;
71
72     while (i > 1 && maxHeap[i / 2].value < x.value)
73     {
74         maxHeap[i] = maxHeap[i / 2];
75         i /= 2;
76     }
77
78     maxHeap[i] = x;
79 }
80
```

```
81 Node maxPop()
82 {
83     Node result = maxHeap[1];
84     Node last = maxHeap[maxSize--];
85
86     int i = 1;
87
88     while (i * 2 <= maxSize)
89     {
90         int child = i * 2;
91
92         if (child + 1 <= maxSize &&
93             maxHeap[child + 1].value > maxHeap[child].value)
94         {
95             child++;
96         }
97
98         if (maxHeap[child].value <= last.value)
99             break;
100
101         maxHeap[i] = maxHeap[child];
102         i = child;
103     }
104
105     if (maxSize >= 0)
106         maxHeap[i] = last;
```

```
108         return result;
109     }
110
111     /* ----- GET VALID MIN ----- */
112
113     Node getMin()
114     {
115         while (minSize > 0 && !active[minHeap[1].id])
116         {
117             minPop();
118         }
119
120         return minPop();
121     }
122
123     /* ----- GET VALID MAX ----- */
124
125     Node getMax()
126     {
127         while (maxSize > 0 && !active[maxHeap[1].id])
128         {
129             maxPop();
130         }
131
132         return maxPop();
```

```
133     }
134
135     /* ----- MAIN ----- */
136
137     int main()
138     {
139         int N, Q;
140
141         scanf("%d %d", &N, &Q);
142
143         ll sum = 0;
144
145         /*
146          * Maximum number of elements that will ever be created:
147          * N initial elements + (N - 1) new elements
148          */
149         int nextID = 0;
150
151         for (int i = 0; i < N; i++)
152         {
153             ll x;
154             scanf("%lld", &x);
155
156             Node node;
157             node.value = x;
```

```
158     node.id = nextID++;
159
160     active[node.id] = 1;
161
162     minPush(node);
163     maxPush(node);
164
165     sum += x;
166 }
167
168 /*
169  * ans[k] = sum of elements after k operations
170  */
171 ll *ans = (ll *)malloc(N * sizeof(ll));
172
173 ans[0] = sum;
174
175 for (int k = 1; k < N; k++)
176 {
177     Node mn = getMin();
178     Node mx = getMax();
179
180     /*
181      * Remove minimum and maximum
182      */
183     active[mn.id] = 0;
```

```

184         active[mx.id] = 0;
185
186         ll difference = mx.value - mn.value;
187
188         /*
189         * Update sum
190         */
191         sum -= mn.value;
192         sum -= mx.value;
193         sum += difference;
194
195         /*
196         * Insert difference
197         */
198         Node newNode;
199         newNode.value = difference;
200         newNode.id = nextID++;
201
202         active[newNode.id] = 1;
203
204         minPush(newNode);
205         maxPush(newNode);
206
207         ans[k] = sum;
208     }
209
210     /*
211     * Answer queries
212     */
213     while (Q--)
214     {
215         int K;
216         scanf("%d", &K);
217
218         printf("%lld\n", ans[K]);
219     }
220
221     free(ans);
222
223     return 0;
224 }

```

Output:

Test Case-1:

```
5 2
3 2 1 5 4
1
13
2
9
```

Test Case-2:

```
5 3
13 12 11 15 14
1
43
2
35
3
15
PS C:\Users\keshav\OneDr
```

Keshav Kumar Rauniyar
CH.SC.U4CSE25236